TECHNICAL REFERENCE DOCUMENT · PART C

Backend Schema Reference

TransitOps — Smart Transport Operations Platform. Complete MongoDB / Mongoose data model: collections, fields, relationships, enums, and the state machines that govern them — derived from the PRD business rules and TRD data model.

PROJECT	DOCUMENT TYPE	BASED ON	VERSION
TransitOps	**Backend Schema**	**PRD v1.0 & TRD v1.0**	**1.0**

PART C — BACKEND SCHEMA

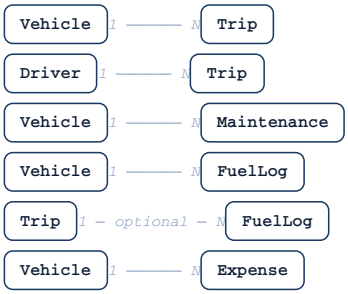
Database Overview & Entity Relationships

Database: **MongoDB**, accessed via **Mongoose ODM**. Seven collections model the full operational lifecycle — one root identity collection (`User`) and six operational collections referencing each other by `ObjectId`.

Collections at a Glance

Collection	Purpose	Key Relationships
User	Login identity + role (RBAC)	Standalone — no refs
Vehicle	Fleet asset master record	Referenced by Trip, Maintenance, FuelLog, Expense
Driver	Driver master record + compliance data	Referenced by Trip
Trip	Dispatch transaction record	References Vehicle + Driver; referenced by FuelLog
Maintenance	Workshop record per vehicle	References Vehicle
FuelLog	Fuel purchase entry	References Vehicle, optionally Trip
Expense	Non-fuel operational cost entry	References Vehicle

Entity Relationship Map



A Trip always has exactly one Vehicle and one Driver (no multi-leg / multi-vehicle trips — PRD §9 assumption). Financial roll-ups (Operational Cost, ROI) join Vehicle → Maintenance + FuelLog + Expense via `vehicle` foreign keys.

PART C — BACKEND SCHEMA

Collection Schemas — Identity & Fleet

User

Login identity — one record per platform user

Field	Type	Notes
name	String	required
email	String	required, **unique**
passwordHash	String	bcrypt hash, never returned in API responses
role	String (enum)	fleet_manager | dispatcher | safety_officer | financial_analyst
createdAt	Date	default: now

Vehicle

Fleet asset master record

Field	Type	Notes
name	String	Vehicle Name / Model
registrationNumber	String	**unique: true, index: true** — enforces BR-1
vehicleType	String (enum)	Truck | Van | Bike | Trailer
maxLoadCapacity	Number	kg — validated against Trip.cargoWeight (BR-5)
odometer	Number	km — incremented on trip completion
acquisitionCost	Number	used as ROI denominator
region	String	supports Dashboard region filter (PRD assumption)
status	String (enum)	available | on_trip | in_shop | retired — default available
createdAt / updatedAt	Date	timestamps: true

Driver

Driver master record + compliance data

Field	Type	Notes
name	String	
licenseNumber	String	**unique: true**
licenseCategory	String (enum)	LMV | HMV
licenseExpiryDate	Date	checked at dispatch time — enforces BR-3
contactNumber	String	
safetyScore	Number	0–100
status	String (enum)	available | on_trip | off_duty | suspended

PART C — BACKEND SCHEMA

Collection Schemas — Operations

Trip

Dispatch transaction record — the busiest schema in the app

Field	Type	Notes
tripCode	String	auto-generated, e.g. TRIP-0001
source	String	
destination	String	
vehicle	ObjectId → Vehicle	ref, must be available at dispatch
driver	ObjectId → Driver	ref, must be available + license valid at dispatch
cargoWeight	Number	validated against vehicle.maxLoadCapacity (BR-5)
plannedDistance	Number	km
actualDistance	Number	filled on completion
fuelConsumed	Number	filled on completion
revenue	Number	used in ROI formula (PRD assumption)
status	String (enum)	draft | dispatched | completed | cancelled

Maintenance

Workshop record — directly drives Vehicle status

Field	Type	Notes
vehicle	ObjectId → Vehicle	ref
description	String	e.g. "Oil Change"
cost	Number	rolls into operational cost
dateStart	Date	
dateEnd	Date	null = still open
status	String (enum)	open | closed

FuelLog

Fuel purchase entry

Field	Type	Notes
vehicle	ObjectId → Vehicle	
trip	ObjectId → Trip	optional
date	Date	
liters	Number	
cost	Number	

Expense

Non-fuel operational cost entry

Field	Type	Notes
vehicle	ObjectId → Vehicle	
expenseType	String (enum)	toll | other
amount	Number	
date	Date	

PART C — BACKEND SCHEMA

State Machines Enforced by the Schema

These transitions are enforced in the controllers (server-side), not just documented — every status field above only ever moves along these paths.

Vehicle.status

available → dispatch trip → on_trip → complete/cancel → available

available → open maintenance → in_shop → close → available*

any → manual admin action → retired (terminal, excluded from dispatch)

**Unless `vehicle.status == retired` at the time maintenance closes — it stays retired.*

Driver.status

available → dispatch trip → on_trip → complete/cancel → available

available → admin action → off_duty / suspended

Trip.status

draft → dispatch → dispatched → complete → completed

draft → cancel → cancelled

dispatched → cancel → cancelled (releases vehicle + driver → available)

Maintenance.status

create → open [vehicle.status → in_shop] → close → closed [vehicle.status → available, unless retired]

Business Rules → Schema Enforcement Map

Rule	Description	Enforced By
BR-1	Vehicle registration number is globally unique	`Vehicle.registrationNumber` — Mongoose `unique: true, index: true` (DB-level constraint)
BR-2	Retired / In Shop vehicles excluded from dispatch lists	Query filter `status: 'available'` on vehicle dropdown fetch
BR-3	Expired-license or Suspended drivers cannot be assigned	`Driver.licenseExpiryDate` + `Driver.status` checked in `dispatchTrip()`
BR-4	A vehicle/driver already On Trip cannot be double-booked	`Vehicle.status` / `Driver.status` checked before dispatch
BR-5	Cargo Weight ≤ Vehicle Max Load Capacity	`Trip.cargoWeight` vs `Vehicle.maxLoadCapacity`, hard 400 block
BR-6 / BR-7 / BR-8	Dispatch / Complete / Cancel ⇒ atomic status sync	`Trip.status` + `Vehicle.status` + `Driver.status` updated together in controller
BR-9 / BR-10	Maintenance open/close ⇒ Vehicle status sync	`Maintenance.status` write triggers `Vehicle.status` write

PART C — BACKEND SCHEMA

Derived Data — KPI & Report Formulas

These are not stored fields — they are computed on read via MongoDB aggregation pipelines (`$match`, `$group`, `$lookup`) across the collections above.

KPI / Report	Formula	Source Collections
Active Vehicles	`count(Vehicle) where status != 'retired'`	Vehicle
Available Vehicles	`count(Vehicle, status='available')`	Vehicle
Vehicles in Maintenance	`count(Vehicle, status='in_shop')`	Vehicle
Active Trips	`count(Trip, status='dispatched')`	Trip
Pending Trips	`count(Trip, status='draft')`	Trip
Drivers On Duty	`count(Driver, status='on_trip')`	Driver
Fleet Utilization %	`(vehicles on_trip / total active vehicles) × 100`	Vehicle
Fuel Efficiency	`trip.actualDistance / trip.fuelConsumed`	Trip
Operational Cost (per vehicle)	`Σ fuelLog.cost + Σ maintenance.cost + Σ expense.amount`	FuelLog, Maintenance, Expense
Vehicle ROI	`(Σ trip.revenue − (Σ maintenance.cost + Σ fuelLog.cost)) / vehicle.acquisitionCost`	Trip, Maintenance, FuelLog, Vehicle

Mongoose-level enforcement is defense-in-depth, not a substitute for controller checks: `unique: true` on `Vehicle.registrationNumber` creates a real MongoDB unique index (BR-1), so a duplicate insert throws `E11000 duplicate key` — the controller should catch this and return a clean `400` instead of a raw MongoDB error.